Programmation sur Processeur Graphique

—

Partie 11 : Streams

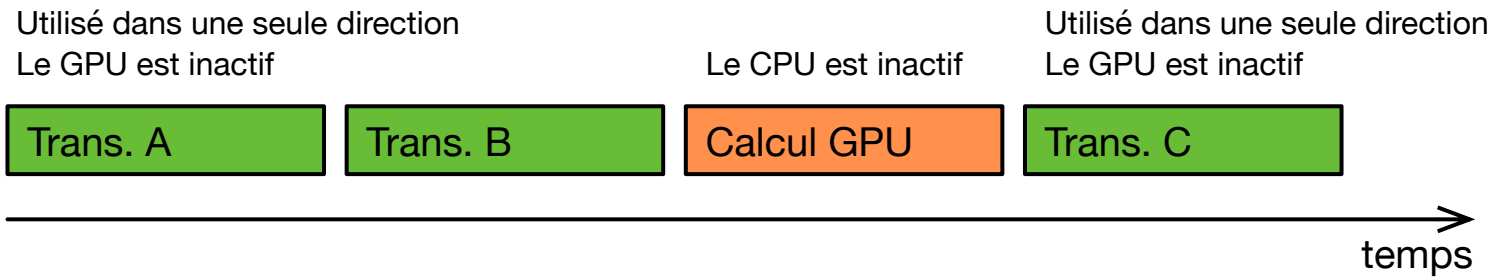
Pierre-Emmanuel Hladik
pierre-emmanuel.hladik@ec-nantes.fr

2 mars 2025

Transfert de données et calcul sérialisés

Exemple `VecAddKernel()`

`cudaMemcpy` sérialise le transfert de données et le calcul GPU



Device Overlap

Exemple VecAddKernel()

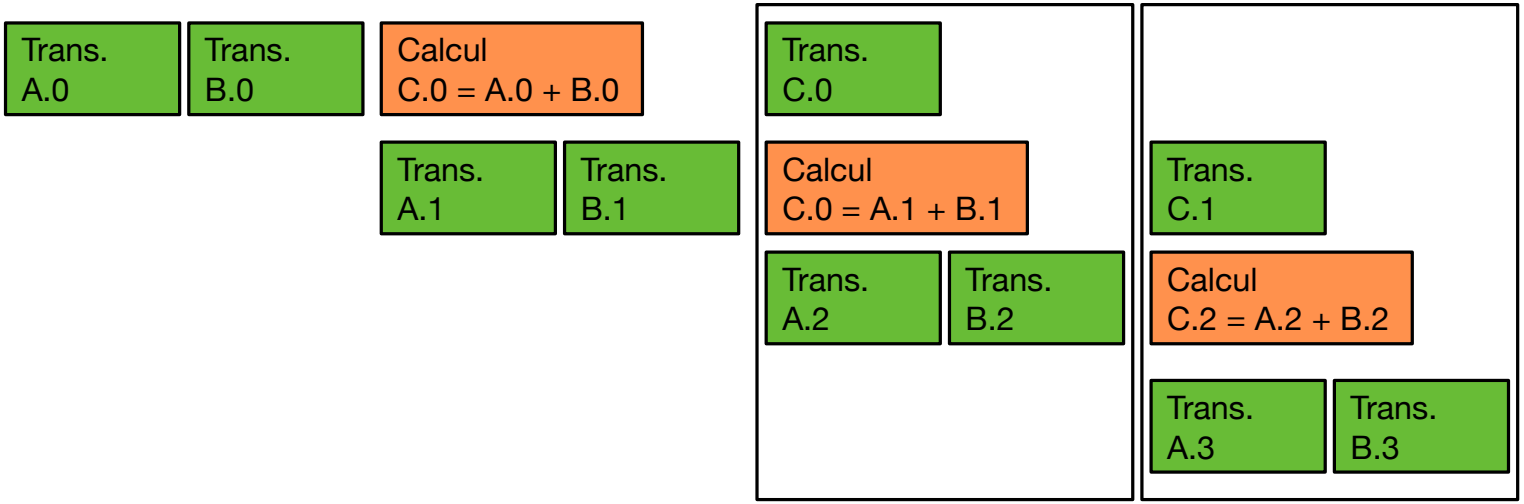
Certains périphériques CUDA permettent l'exécution simultanée d'un kernel tout en copiant des données entre le périphérique et la mémoire hôte.

```
int dev_count;
cudaDeviceProp prop;
cudaGetDeviceCount( &dev_count);

for (int i = 0; i < dev_count; i++) {
        cudaGetDeviceProperties(&prop, i);
        if (prop.deviceOverlap)
        ...
```

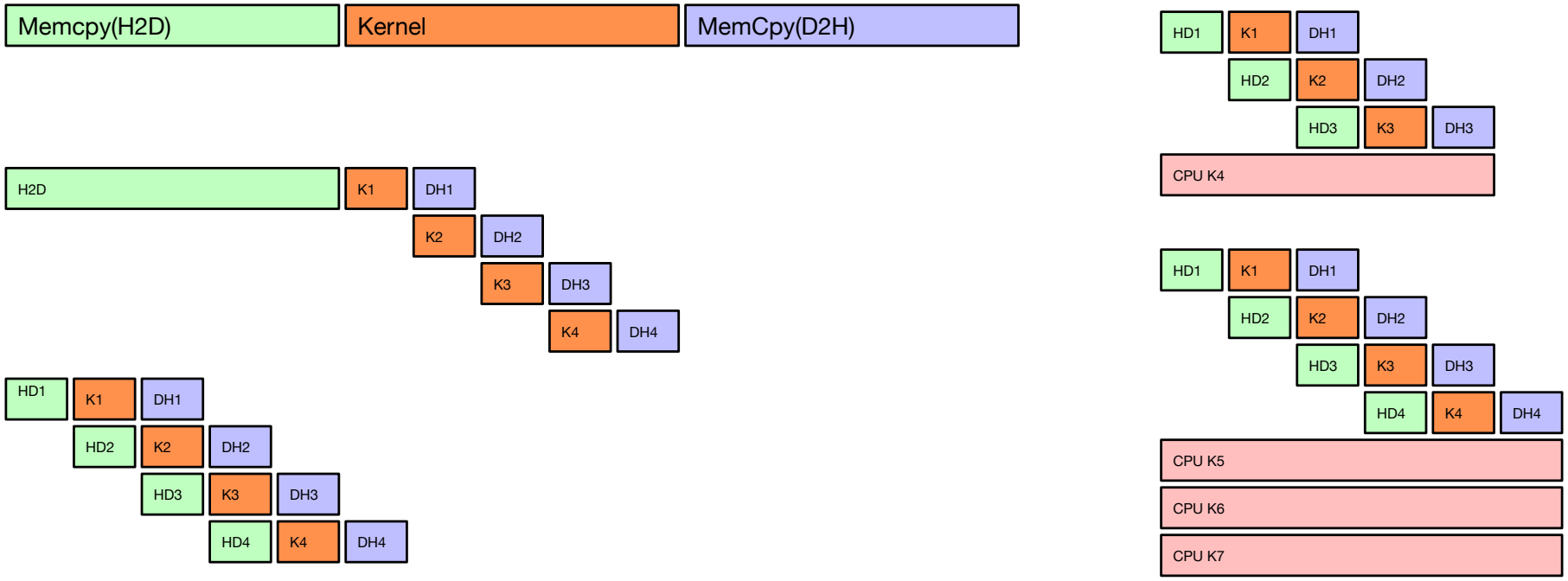
Fonctionnement idéale en pipeline

Exemple `VecAddKernel()`

- Diviser les grands vecteurs en segments
- Superposer les transferts et les calculs des segments adjacents



Concurrence par le biais du pipeling



Synchronicité en CUDA

Synchrone vs asynchrone

Tous les appels CUDA sont soit synchrones, soit asynchrones par rapport à l'hôte :

- Synchrone : mise en file d'attente du travail et attente de sa terminaison
- Asynchrone : mise en file d'attente du travail et retour immédiat

Le lancement d'un kernel est asynchrone et se recouvre (*overlap*) automatiquement avec l'hôte

Les streams CUDA

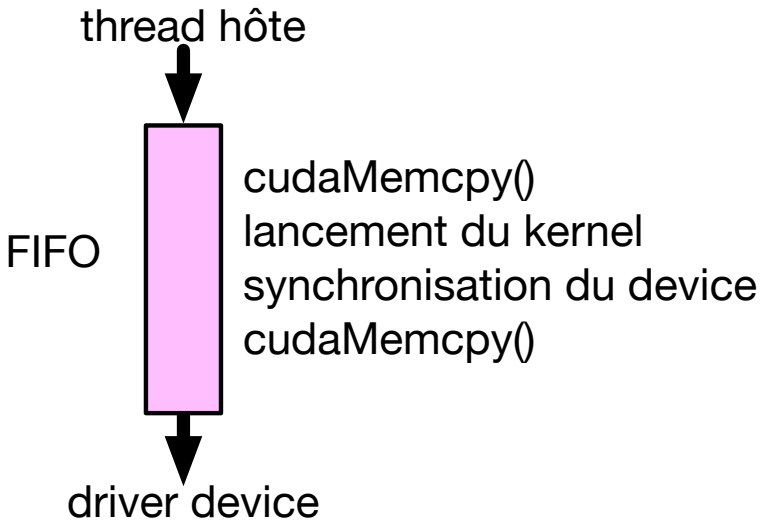
CUDA prend en charge l'exécution parallèle des kernels et de cudaMemcpy avec des **streams** :

- Chaque *stream* est une file d'attente d'opérations sur le device (lancements de kernels et appels à `cudaMemcpy`) :
  - L'hôte place le travail dans la file d'attente et continue immédiatement
  - Le device planifie le travail à partir des streams lorsque les ressources sont libres
- Les opérations (tâches) de différents `streams` peuvent être exécutées en parallèle (parallélisme des tâches) : Mem CPU->GPU // Mem CPU-> GPU // kernel

Ordre execution d'un stream

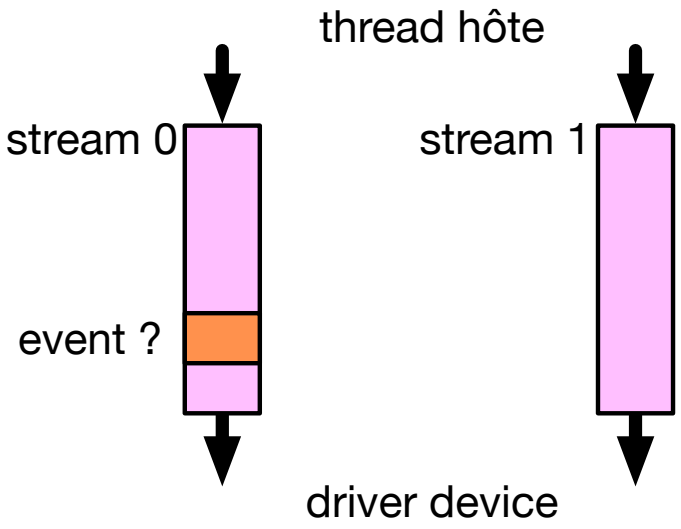
Les demandes faites par l'hôte sont placées dans des files d'attente de type « premier entré, premier sorti » (FIFO) :

- Les files d'attente sont lues et traitées de manière asynchrone par le pilote et le device.
- Le pilote s'assure que les opérations dans une file d'attente sont traitées en séquence. Par exemple, les copies de mémoire se terminent avant le lancement du noyau, etc.



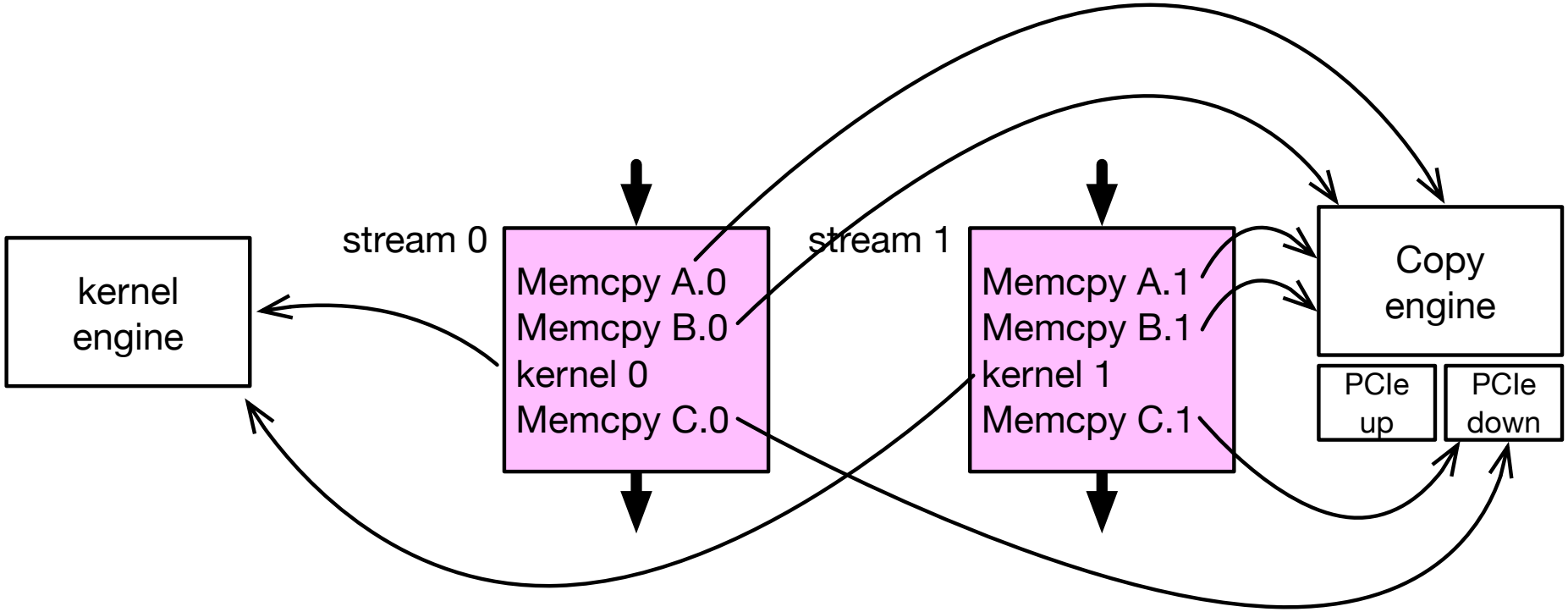
Execution de plusieurs streams

- Pour permettre le parallèlisme des transferts mémoire et de l'exécution des noyau, il faut utiliser plusieurs *streams*



- Les événements CUDA permettent au thread hôte d'interroger et de se synchroniser avec des files d'attente individuelles (c'est-à-dire des *streams*)

Vue conceptuelle de l'exécution des streams



- Les traitements sont dépliés dans chaque FIFO (kernel, copy up, copy down)
- ... **et l'ordre d'exécution est maintenu dans chaque stream**

Gérer les *streams*

- Déclaration d'un gestionnaire de *stream* :

```
cudaStream_t stream;
```

- Allocation d'un *stream* :

```
cudaStreamCreate(&stream);
```

- Désallocation d'un *stream*

```
cudaStreamDestroy(stream);
```

Synchronise l'hôte jusqu'à ce que le travail du *stream* soit terminé

Placer un travail dans un *stream*

- Placer un kernel (4e paramètre) :

```
kernel<<< blocks , threads, smem, stream>>>();
```

- Placer une copie de données :

```
cudaMemcpyAsync( dst, src, size, dir, stream);
```

stream par défaut

- Sauf indication contraire, tous les appels sont placés dans un *stream* par défaut, souvent appelé « *stream 0* »
- Le *stream 0* a des règles de synchronisation spéciales :
  - Synchrone avec tous les streams : les opérations dans le *stream 0* ne peuvent pas chevaucher les autres streams

```
cudaStream_t stream1;
cudaStreamCreate(&stream1);

foo<<<blocks,threads>>>(); // Stream par defaut
foo<<<blocks,threads,0,stream1>>>(); // pas de concurrence dans ce cas

cudaStreamDestroy(stream1);
```

Exemple de code pour gérer les streams

```
// Declaration
cudaStream_t stream1, stream2;

// Instanciation
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

// Affectation des kernels aux streams
foo<<<blocks,threads,0,stream1>>>();
foo<<<blocks,threads,0,stream2>>>();

// Destrcution des streams
cudaStreamDestroy(stream1);
cudaStreamDestroy(stream2);
```

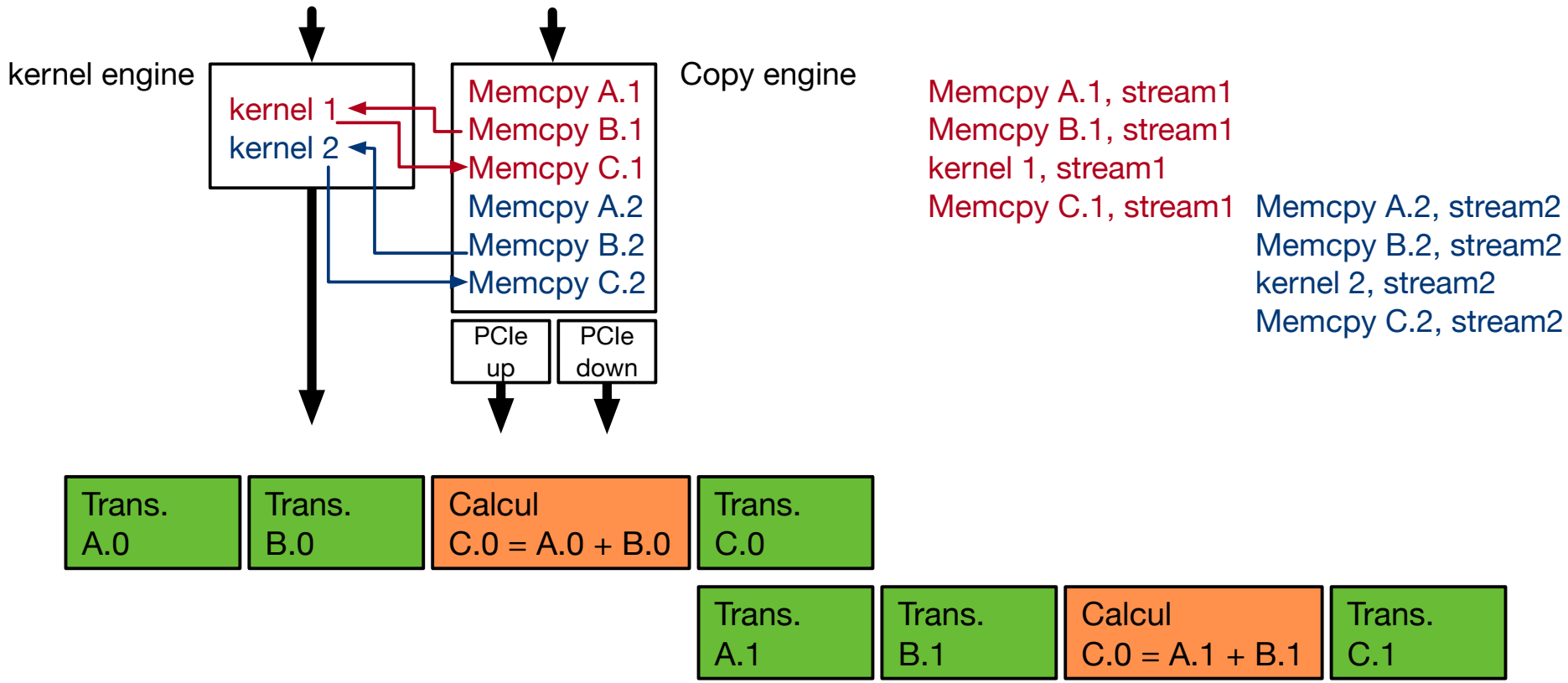
Copies concurrentes de la mémoire

- `cudaMemcpy(...)`
  - Le transfert mémoire est fait dans le `stream 0` par défaut
  - Est synchrone (i.e. doit être terminé avant son retour)
- `cudaMemcpyAsync(..., &stream)`
  - Place le transfert mémoire dans le stream et retourne immédiatement
- La concurrence s'obtient si :
  - les transferts se font dans un stream qui n'est pas celui par défaut
  - les copies sont asynchrones
  - un seul transfert par direction à la fois
  - la mémoire de l'hôte est *pinned*

Code hôte pour un simple Multi-Stream

```
cudaStream_t stream1, stream2;

cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

float *d_A1, *d_B1, *d_C1; // device memory for stream 1
float *d_A2, *d_B2, *d_C2; // device memory for stream 2

// cudaMalloc() calls for d_A1, d_B1, d_C1, d_A2, d_B2, d_C2 go here
// h_A, h_B and h_C : vector on host

for (int i=0; i<n; i+=2*SegSize) { // n vector size, SegSize size of the segment (divisor of n)

        // Gestion stream 1
        cudaMemcpyAsync(d_A1, h_A+i, SegSize*sizeof(float), cudaMemcpyHostToDevice, stream1);
        cudaMemcpyAsync(d_B1, h_B+i, SegSize*sizeof(float), cudaMemcpyHostToDevice, stream1);
        vecAdd<<<SegSize/256, 256, 0, stream1>>>(d_A1, d_B1, d_C1);
        cudaMemcpyAsync(h_C+i, d_C1, SegSize*sizeof(float),cudaMemcpyDeviceToHost, stream1);

        // Gestion stream 2
        cudaMemcpyAsync(d_A2, h_A+i+SegSize, SegSize*sizeof(float), cudaMemcpyHostToDevice, stream2);
        cudaMemcpyAsync(d_B2, h_B+i+SegSize, SegSize*sizeof(float), cudaMemcpyHostToDevice, stream2);
        vecAdd<<<SegSize/256, 256, 0, stream2>>>(d_A2, d_B2, d_C2);
        cudaMemcpyAsync(h_C+i+SegSize, d_C2, SegSize*sizeof(float),cudaMemcpyDeviceToHost, stream2);
}
```

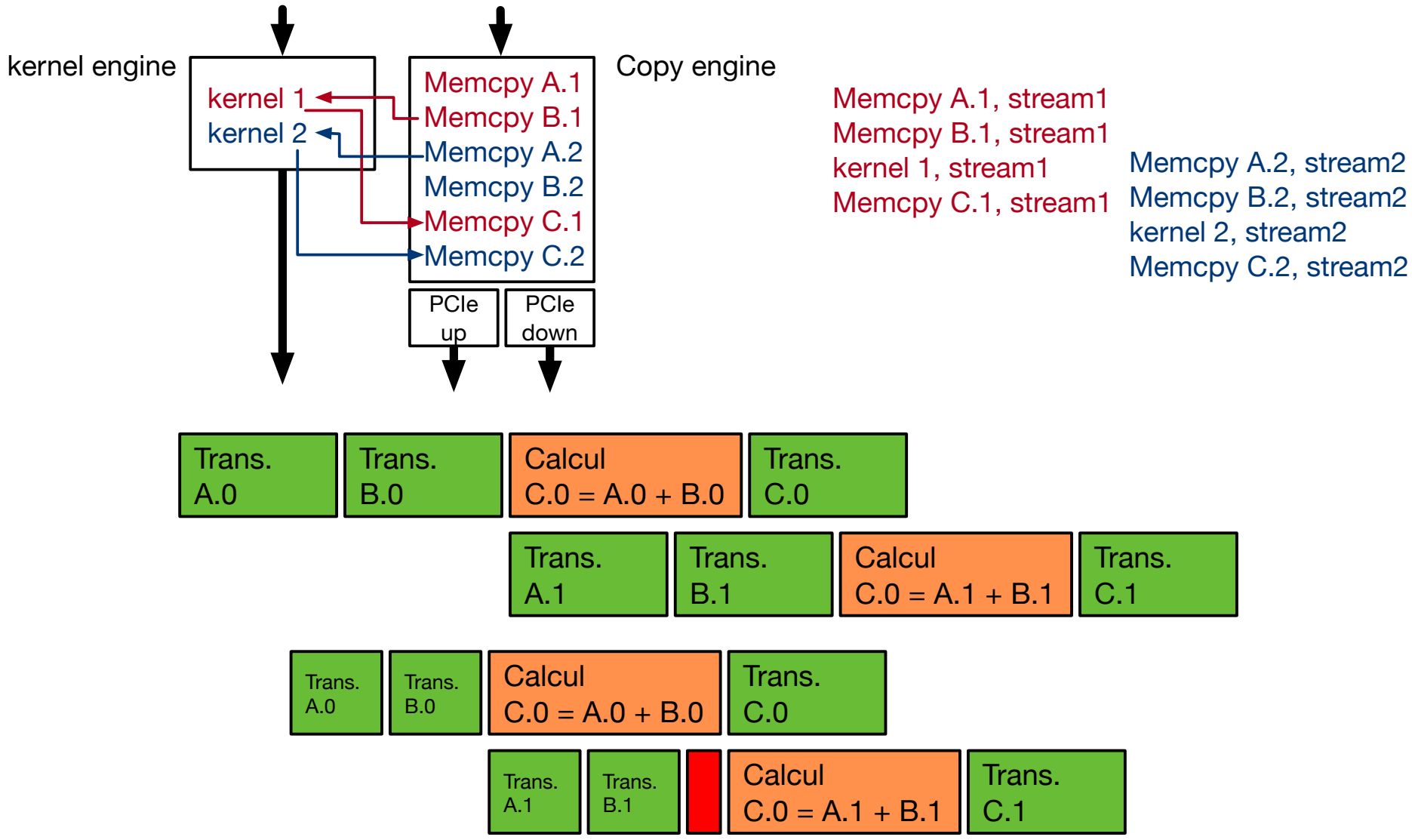
Exécution des streams : mauvaise synchronisation



Code hôte avec une meilleure utilisation du Multi-Stream

```
cudaStream_t stream1, stream2;

cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

float *d_A1, *d_B1, *d_C1; // device memory for stream 1
float *d_A2, *d_B2, *d_C2; // device memory for stream 2

// cudaMalloc() calls for d_A1, d_B1, d_C1, d_A2, d_B2, d_C2 go here
// h_A, h_B and h_C : vector on host

for (int i=0; i<n; i+=2*SegSize) { // n vector size, SegSize size of the segment (divisor of n)

        // Entrelacement des copies memoire
        cudaMemcpyAsync(d_A1, h_A+i, SegSize*sizeof(float),cudaMemcpyHostToDevice, stream1);
        cudaMemcpyAsync(d_B1, h_B+i, SegSize*sizeof(float),cudaMemcpyHostToDevice, stream1);

        cudaMemcpyAsync(d_A2, h_A+i+SegSize, SegSize*sizeof(float),cudaMemcpyHostToDevice, stream2);
        cudaMemcpyAsync(d_B2, h_B+i+SegSize, SegSize*sizeof(float),cudaMemcpyHostToDevice, stream2);

        vecAdd<<<SegSize/256, 256, 0, stream1>>>(d_A1, d_B1, d_C1);
        vecAdd<<<SegSize/256, 256, 0, stream2>>>(d_A2, d_B2, d_C2);

        cudaMemcpyAsync(h_C+i, d_C1, SegSize*sizeof(float),cudaMemcpyDeviceToHost, stream1);
        cudaMemcpyAsync(h_C+i+SegSize, d_C2, SegSize*sizeof(float),cudaMemcpyDeviceToHost, stream2);
}
```

Exemple d'exécution des streams entrelacées



Synchronisation

- Tout synchroniser : bloque l'hôte jusqu'à ce que tous les appels CUDA émis soient terminés

```
cudaDeviceSynchronize()
```

- Synchroniser l'hôte avec un flux spécifique : bloque l'hôte jusqu'à ce que tous les appels CUDA émis dans le stream soient terminés

```
cudaStreamSynchronize ( stream)
```

- Synchronisation de l'hôte ou des périphériques à l'aide d'événements

Mémoire unifiée et stream

Préfetching de données dans la mémoire unifiée CUDA :

- L'objectif principal est d'éviter les défauts de page
- Fournit un mécanisme pour améliorer les performances de l'application en établissant la localité des données
- Utilisé pour migrer les données vers un device ou l'hôte et mapper les tables de pages sur le processeur avant qu'il ne commence à utiliser les données.

API CUDA pour le prefetching de données

```
__host__ cudaError_t cudaMemPrefetchAsync ( const void* devPtr, size_t
count, int dstDevice, cudaStream_t stream = 0 )
```

- `devPtr` : Adresse de mémoire unifiée de la région de mémoire pour le prefectch
- `count` : Taille en octets de la région du prefectch
- `dstDevice` : Processeur de destination
- `stream` : stream CUDA

Remarques :

- Le prefetching est une opération asynchrone par rapport au device, mais l'appel peut ne pas être totalement asynchrone par rapport à l'hôte
- Le processeur de destination doit être un device ID ou cudaCpuDeviceId valide
- Si le processeur de prefetch est un GPU, la propriété du device cudaDevAttrConcurrentManagedAccess doit être différente de zéro

Code hôte avec une meilleure utilisation du Multi-Stream

```
// data must've been allocated with cudaMallocManaged( (void**) &data, N);

void function(int * data, cudaStream_t stream) {
        init(data, N);

        // Prefetch to the device cudaStreamSynchronize(stream);
        cudaMemPrefetchAsync(data, N * sizeof(int), myGpuId, stream);

        my_kernel<<<..., stream>>>(data, ...);

        // Prefetch to the host cudaStreamSynchronize(stream);
        cudaMemPrefetchAsync(data, N * sizeof(int), cudaCpuDeviceId, stream);

        hostFunction(data, N);
}
```

Un peu de pratique avec le TD 11

Objectifs

- Utilisation des streams